

Generative Adversarial Networks

Ref: Raschka, Sebastian; Liu, Yuxi (Hayden); Mirjalili, Vahid. Machine Learning with PyTorch and Scikit-Learn: Develop machine learning and deep learning models with Python.

Outline

Introduction to generative models, Autoencoders

Implementing a GAN model using Pytorch in Google collab

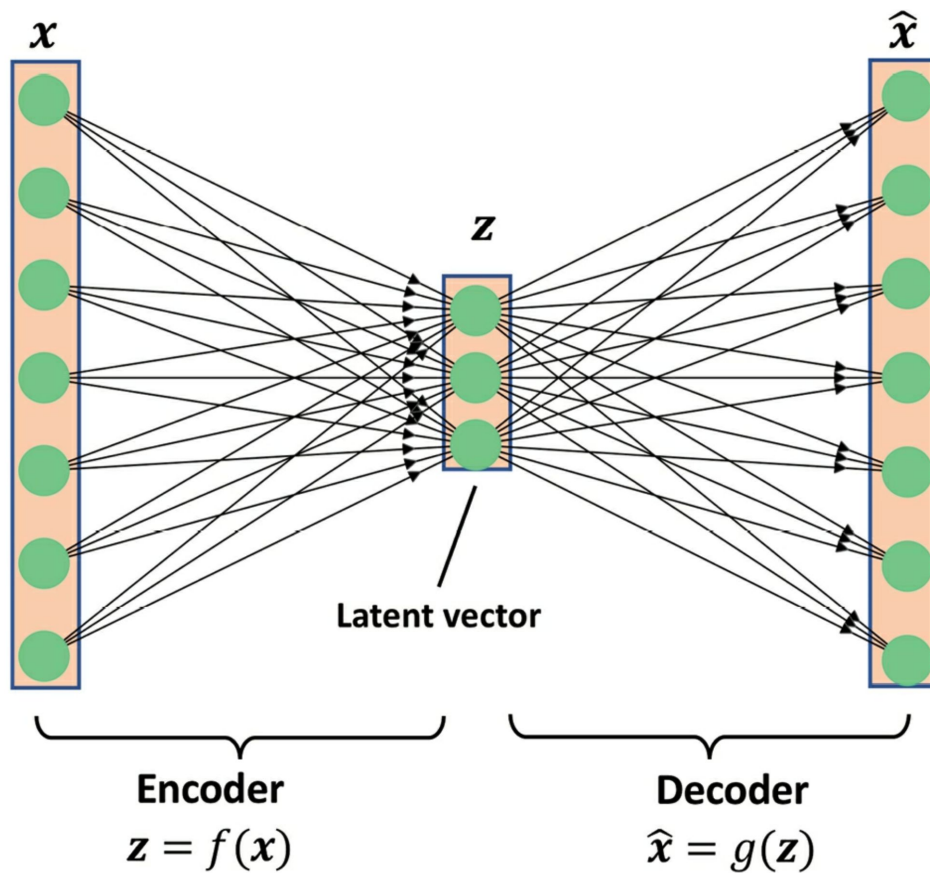
Improving the quality of synthetic data generated using GANs

Different types of GANs and applications

Autoencoder

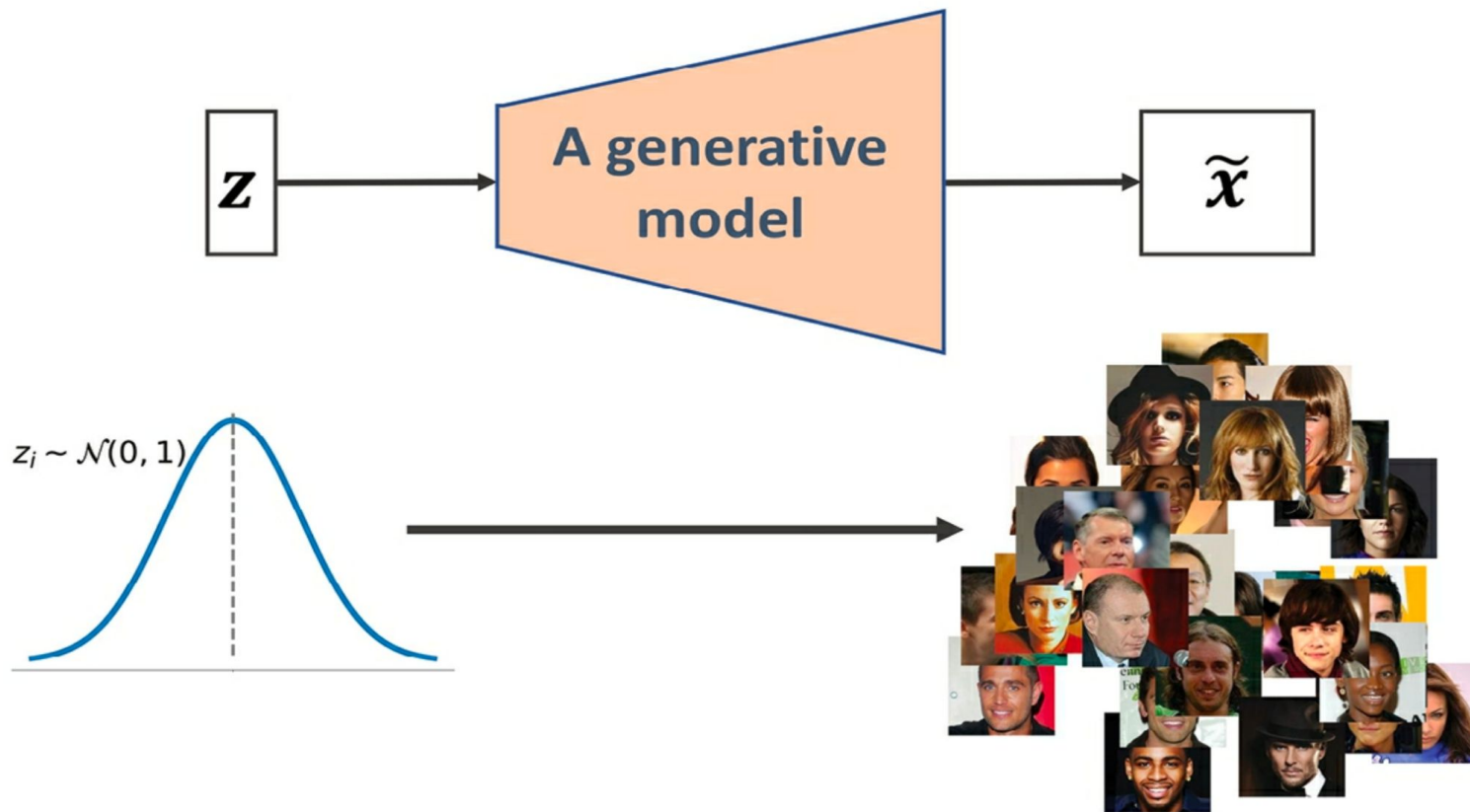
Deterministic model

Undercomplete?
Overcomplete?

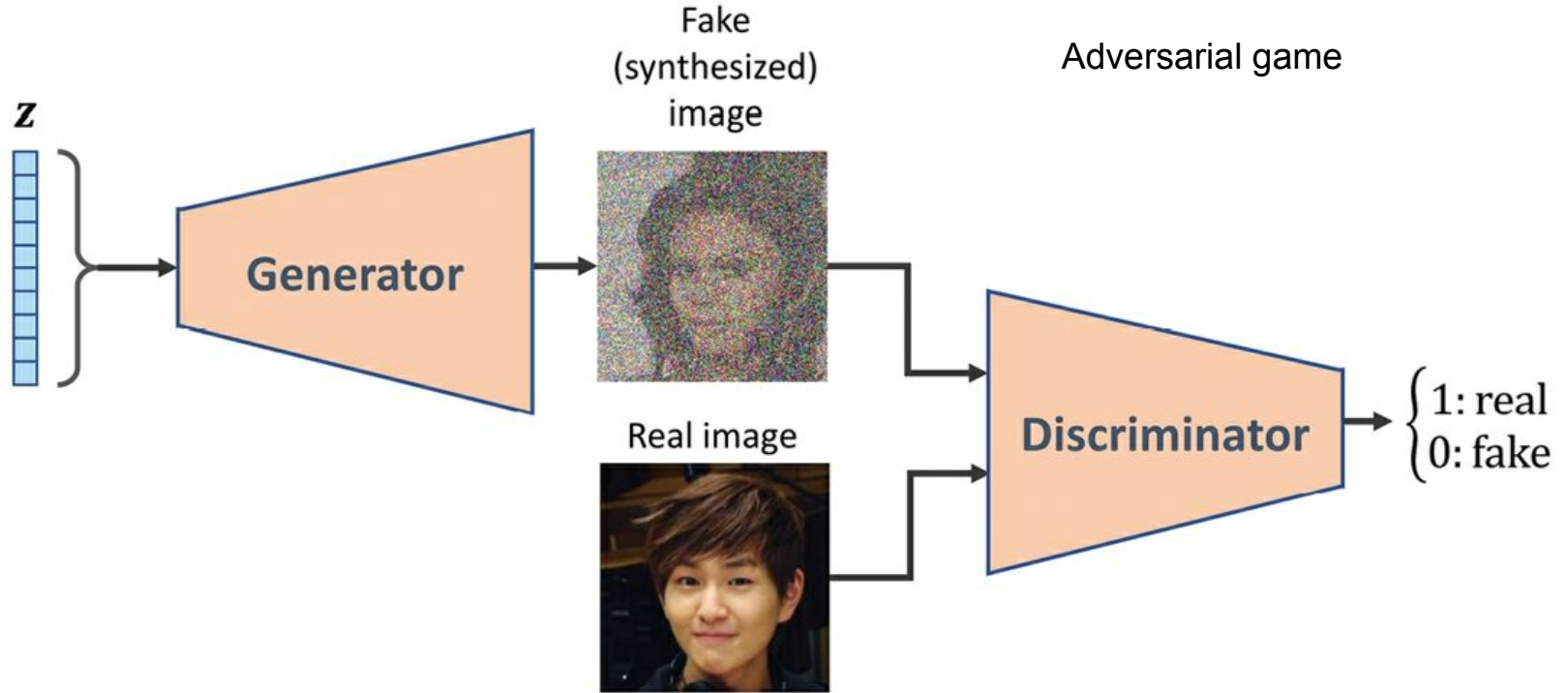


Generative models

Variational autoencoder
(VAE)



Generative Adversarial Network (GAN)



The objective function of GANs

$$V(\theta^{(D)}, \theta^{(G)}) = E_{x \sim p_{data}(x)} [\log D(x)] + E_{z \sim p_z(z)} [\log (1 - D(G(z)))]$$

Value function

$$\min_G \max_D V(\theta^{(D)}, \theta^{(G)})$$

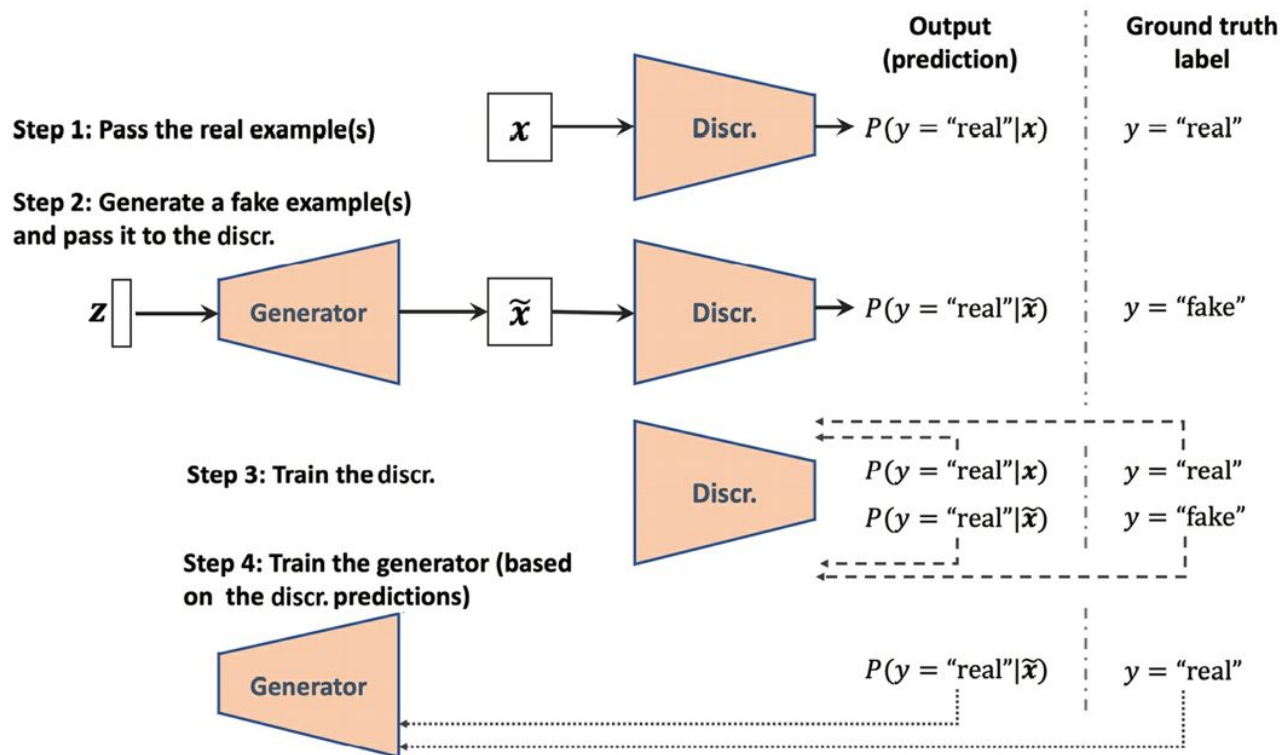
1. Fix (freeze) G() ?
2. Fix (freeze) D() ?

maximize $V(\theta^{(D)}, \theta^{(G)})$.

minimize $V(\theta^{(D)}, \theta^{(G)})$

Steps in GANs

Ground truth labels = $\begin{cases} 1: & \text{for real images, i.e., } \mathbf{x} \\ 0: & \text{for outputs of } G, \text{ i.e., } G(\mathbf{z}) \end{cases}$



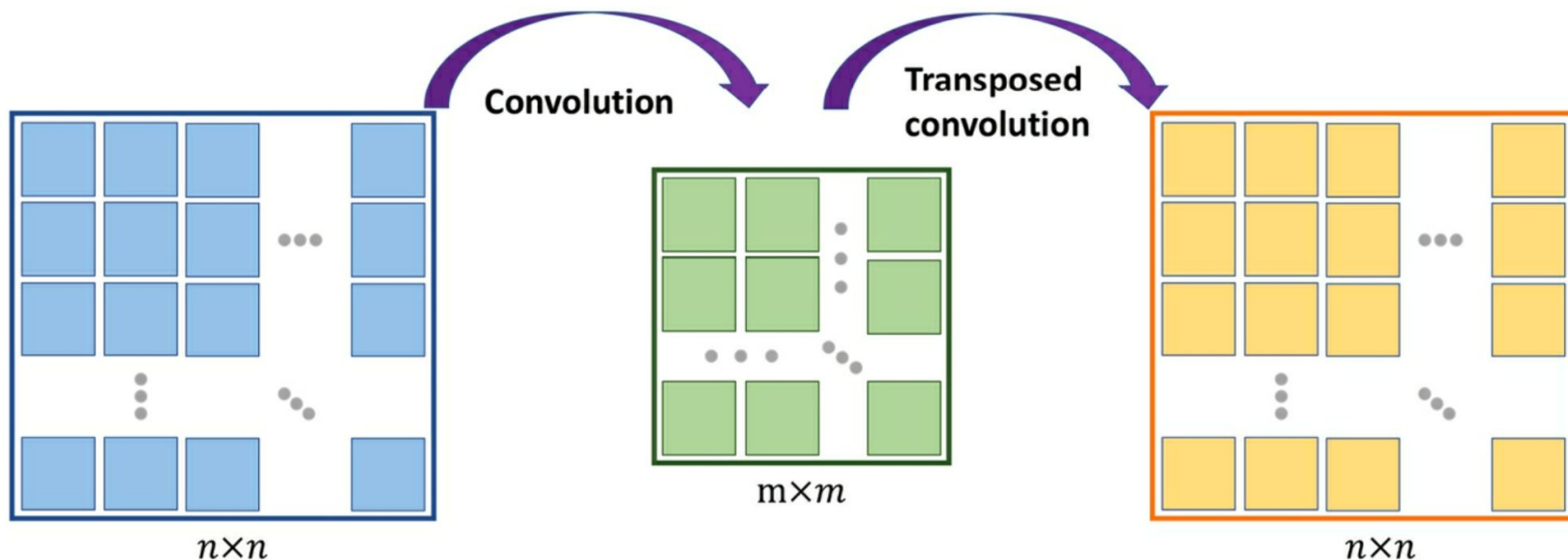
GAN Lab

<https://poloclub.github.io/ganlab/>

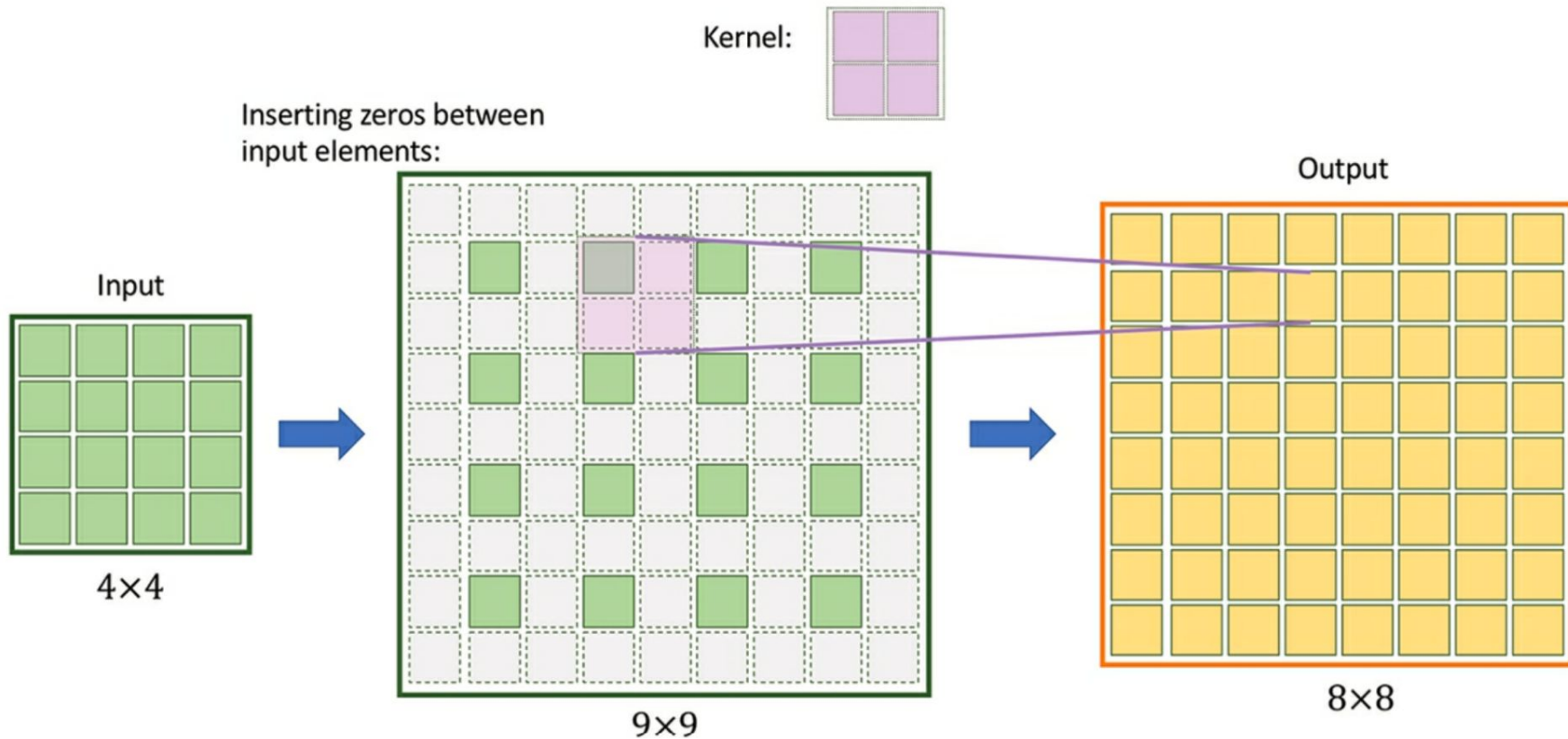
Shall we implement our first GAN model together?

Deep convolutional GAN (DCGAN)

Need to use Transposed Convolution (fractionally strided convolution, deconvolution)

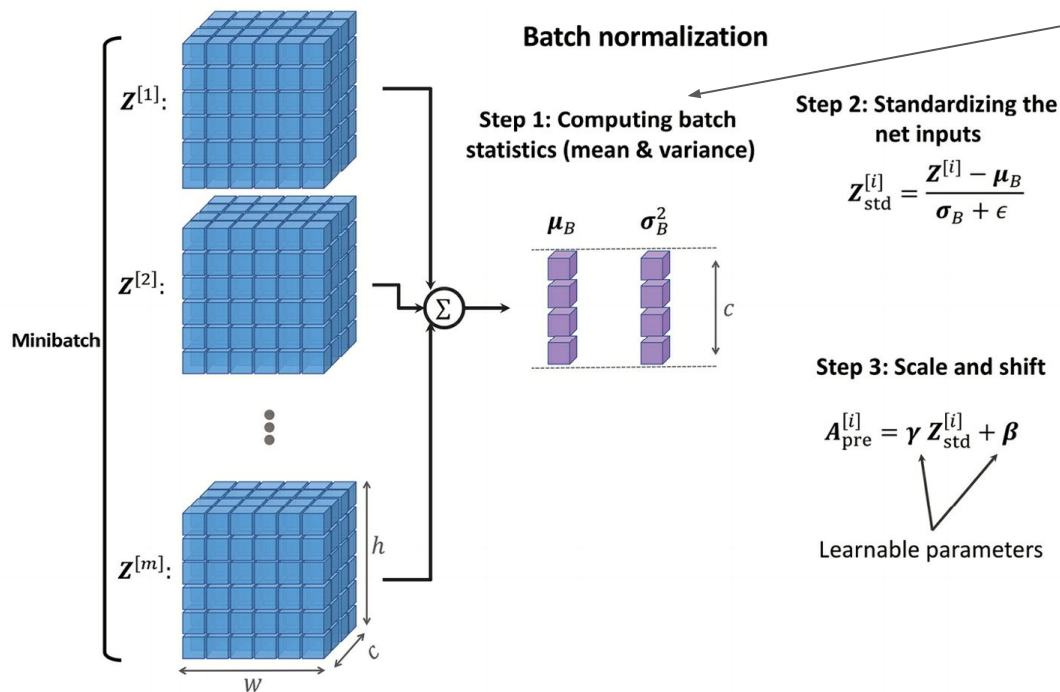


Transpose convolution



Another useful piece: batch normalization

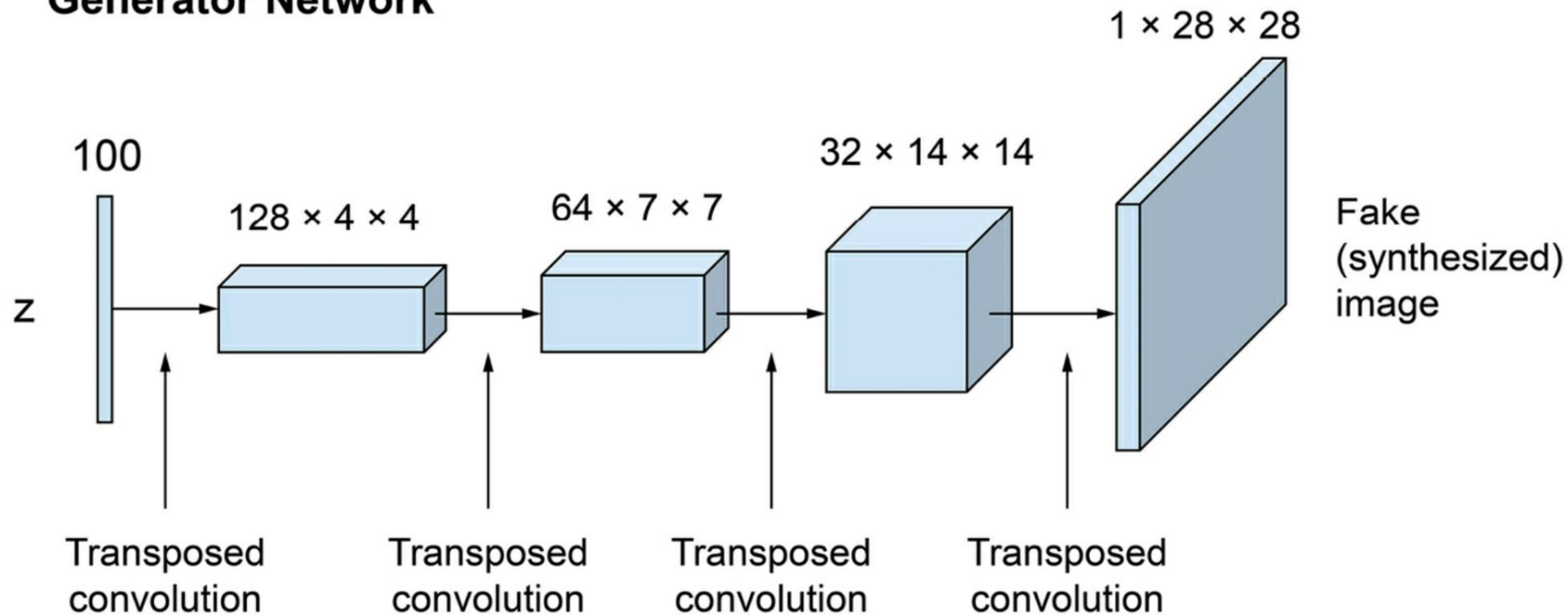
For faster and better convergence.



$$\boldsymbol{\mu}_B = \frac{1}{m \times h \times w} \sum_{i,j,k} \mathbf{Z}^{[i,j,k]}$$
$$\sigma_B^2 = \frac{1}{m \times h \times w} \sum_{i,j,k} (\mathbf{Z}^{[i,j,k]} - \boldsymbol{\mu}_B)^2$$

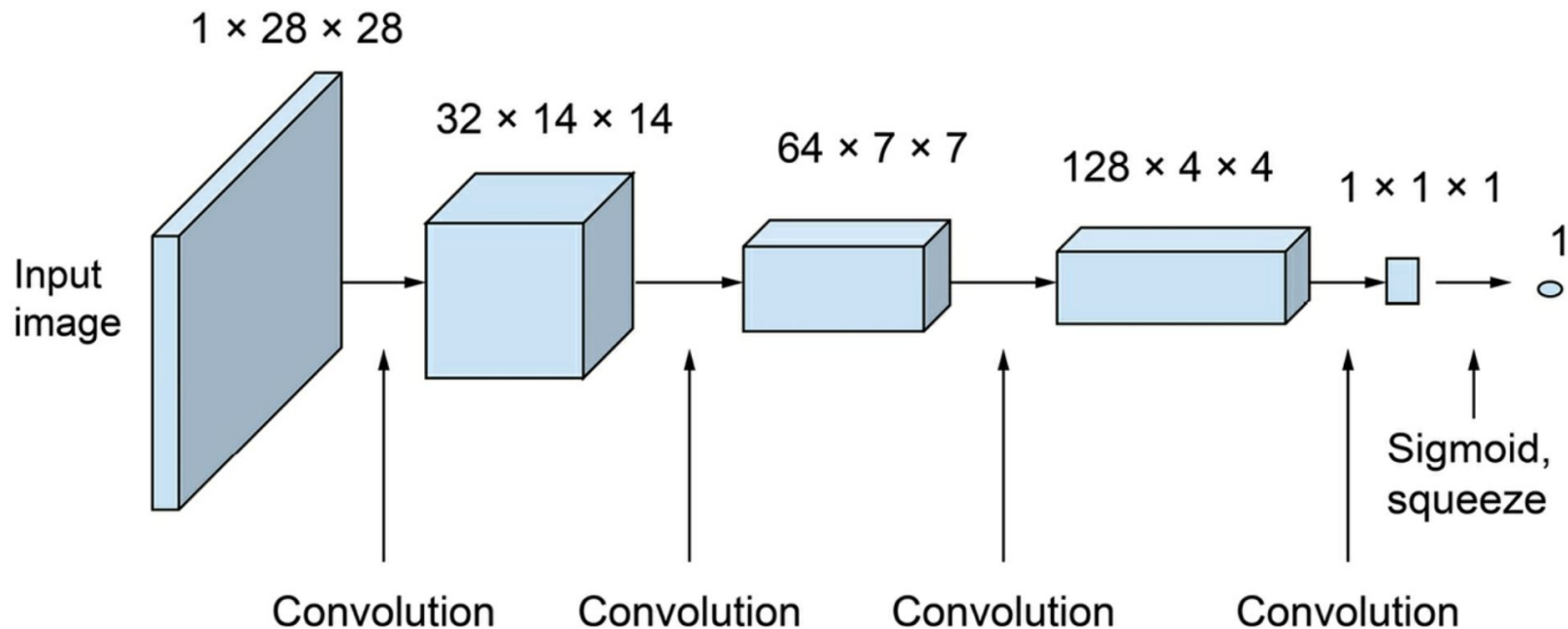
DCGAN: G()

Generator Network



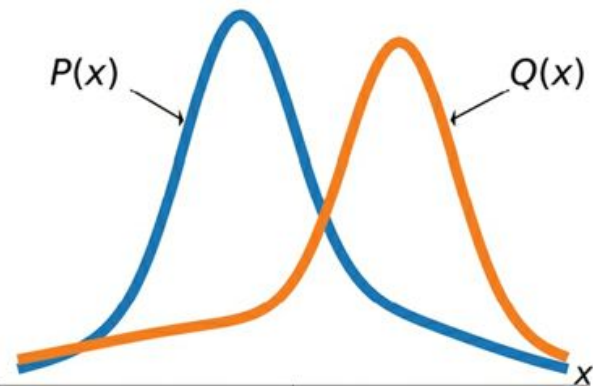
DCGAN: D()

Discriminator Network

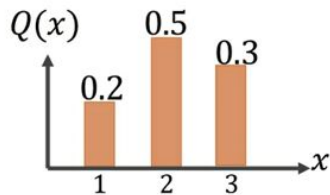
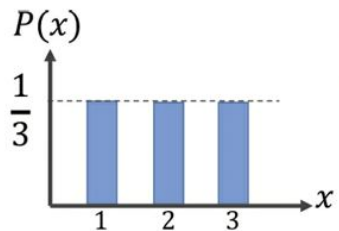


Dissimilarity measures between two distributions

Measures	Formulation
Total variation (TV)	$TV(P, Q) = \sup_x P(x) - Q(x) $
Kullback-Leibler (KL) divergence	$KL(P Q) = \int P(x) \log \frac{P(x)}{Q(x)} dx$
Jensen-Shannon (JS) divergence	$JS(P, Q) = \frac{1}{2} \left(KL \left(P \frac{P+Q}{2} \right) + KL \left(Q \frac{P+Q}{2} \right) \right)$
Earth mover's (EM) distance	$EM(P, Q) = \inf_{\gamma \in \Pi(P, Q)} E_{(u, v) \in \gamma} (\ u - v\)$



Dissimilarity measures between two distributions



Total variation:

$$TV(P, Q) = \sup_x \left\{ \left| \frac{1}{3} - 0.2 \right|, \left| \frac{1}{3} - 0.5 \right|, \left| \frac{1}{3} - 0.3 \right| \right\} = 0.167$$

KL divergence:

$$KL(P||Q) = 0.33 \log\left(\frac{0.33}{0.2}\right) + 0.33 \log\left(\frac{0.33}{0.5}\right) + 0.33 \log\left(\frac{0.33}{0.3}\right) = 0.101$$

$$KL(Q||P) = 0.2 \log\left(\frac{0.2}{0.33}\right) + 0.5 \log\left(\frac{0.5}{0.33}\right) + 0.33 \log\left(\frac{0.3}{0.33}\right) = 0.099$$

JS divergence:

$$P_m \rightarrow \left[\frac{0.33 + 0.2}{2}, \frac{0.33 + 0.5}{2}, \frac{0.33 + 0.3}{2} \right] = [0.26, 0.42, 0.32]$$

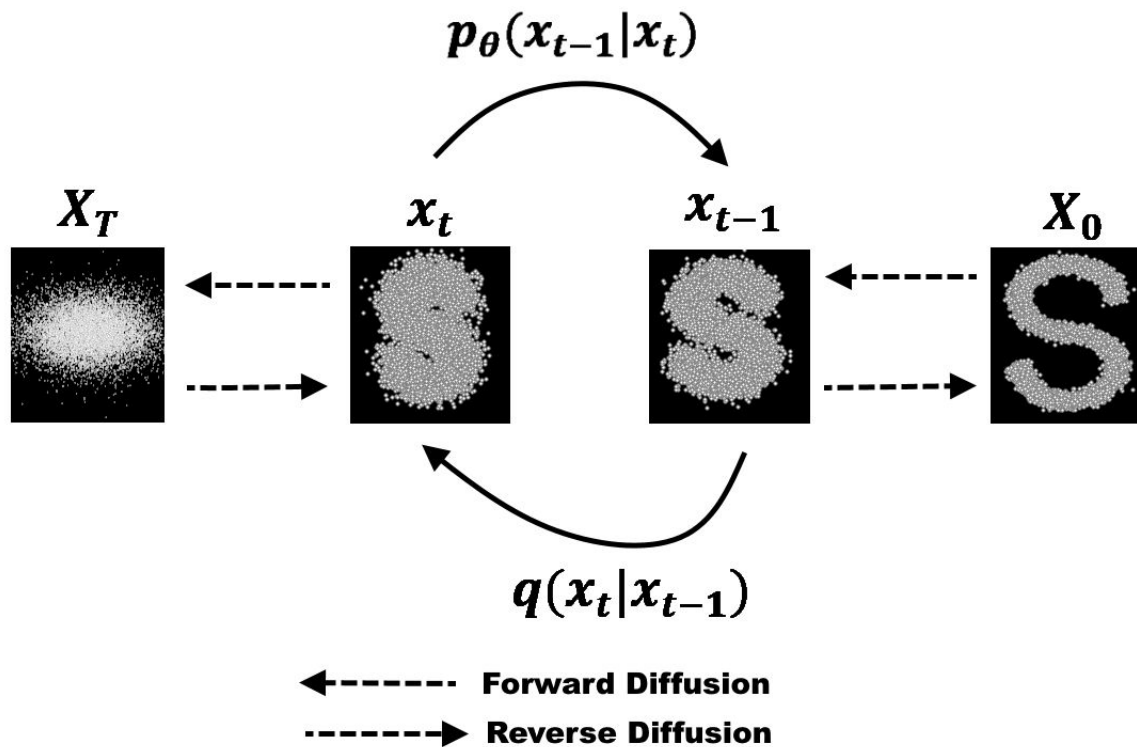
$$\left. \begin{array}{l} KL(P||P_m) = 0.0246 \\ KL(Q||P_m) = 0.0246 \end{array} \right\} \rightarrow JS(P||Q) = 0.0248$$

EM distance:

$$EM(P, Q) = (0.33 - 0.2) + (0.33 - 0.3) = 0.16$$

<https://github.com/hindupuravinash/the-gan-zoo>

Diffusion model



Some code to test

<https://github.com/lucidrains/denoising-diffusion-pytorch>

<https://github.com/CompVis/latent-diffusion>